Міністерство освіти і науки України
Національний університет "Львівська політехніка"
Кафедра ЕОМ




Звіт

з лабораторної роботи № 3

з дисципліни: «АВТОМАТИЗОВАНЕ ПРОЕКТУВАННЯ КОМП'ЮТЕРНИХ СИСТЕМ»

на тему «Реалізація серверної (апаратної) та клієнтської (програмної) частин гри»

Варіант №21

**Виконала:**
ст. гр. КІ-404
Хилинська Л. В.
**Прийняв:**
Федак П. Р

Львів – 2024

**Порядок виконання роботи:**

1. Develop Server and Client.
2. Required steps.

Варіант завдання:

| Student number | Game | config format |
|---|---|---|
| **21** | tik-tac-toe 3x3 | INI |

Опис гри "tik-tac-toe" для Arduino та програмного клієнта

Ця програма реалізує класичну гру "tik-tac-toe и" з підтримкою трьох режимів:

1. Людина проти людини — двоє гравців по черзі роблять ходи через одну програму.
2. Людина проти штучного інтелекту — один гравець грає проти програми, яка використовує алгоритм для вибору ходів.
3. Штучний інтелект проти штучного інтелекту — два штучних інтелекти грають один проти одного без участі людини.

Принцип роботи гри:

- Модель (Model):Апаратна частина на основі Arduino, що відповідає за перевірку коректності ходів та визначення результату гри (перемога, нічия чи продовження).
- Контролер (Controller):Програмний клієнт, який взаємодіє з Arduino через UART, передає дані про ходи, отримує стан гри та керує її логікою.
- Відображення (View):Інтерфейс користувача, що показує поточний стан гри у текстовому вигляді та дозволяє робити ходи.

Архітектура та взаємодія:

1. Комунікація через UART:
    - Клієнт передає хід на сервер (Arduino).
    - Сервер обробляє хід, оновлює модель гри та надсилає новий стан гри клієнту.
    - Контролер клієнта аналізує отримані дані та оновлює відображення гри.
2. Перевірка результату гри:
    - Сервер (Arduino) визначає результат (перемога чи нічия) та повідомляє про це клієнту.
    - Якщо один із гравців виграє або всі клітинки заповнені, гра завершується.
3. Патерн MVC (Model-View-Controller):
    - Model: Arduino обробляє логіку гри та результати.
    - View: текстовий інтерфейс клієнта відображає поточний стан гри.
    - Controller: програмний клієнт керує взаємодією між користувачем та апаратною частиною.

## Виконання роботи:

Архітектурна діаграма проекту демонструє взаємодію між компонентами клієнтської та серверної частин гри "tik-tac-toe", розробленої з використанням шаблону проектування MVC (Model-View-Controller):

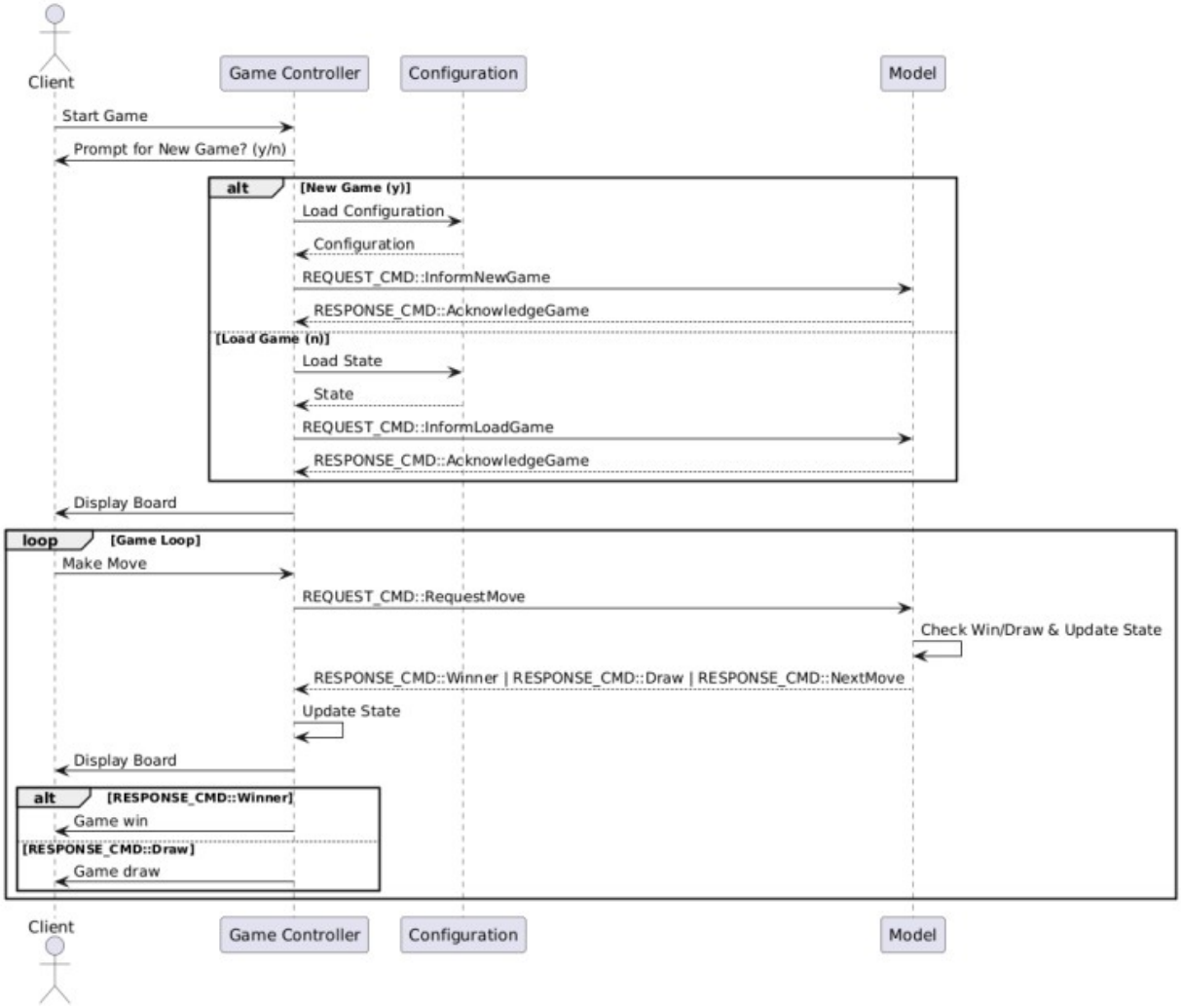



Рис.1. Архітектура гри взаємодії між сервером та клієнтом

Опис кроків діаграми

1. Початок гри (Start Game):
   - Клієнт (SW) ініціює гру, звертаючись до Контролера (Controller).
   - Контролер запитує у користувача, чи бажає він почати нову гру (Prompt for New Game?).
2. Нова гра (New Game, y):

- Якщо користувач обирає нову гру (y), Контролер завантажує конфігурацію через компонент Configuration (CONF).
- Контролер надсилає команду REQUEST_CMD::InformNewGame до Model (HW), сповіщаючи про початок нової гри.
- Model (HW) підтверджує отримання запиту, повертаючи RESPONSE_CMD::AcknowledgeGame.

**3.** Завантаження гри (Load Game, n):

- Якщо користувач обирає завантажити попередню гру (n), Контролер запитує стан гри з компонента Configuration (CONF).
- Контролер надсилає команду REQUEST_CMD::InformLoadGame до Model (HW) для відновлення попереднього стану гри.
- Model (HW) підтверджує отримання запиту та повертає RESPONSE_CMD::AcknowledgeGame.

**4.** Відображення дошки (Display Board):

- Після завантаження конфігурації або стану гри Контролер передає Клієнту (SW) актуальний вигляд ігрової дошки для відображення.

**5.** Ігровий цикл (Game Loop):

- У рамках кожного ходу:
    - Клієнт (SW) передає свій хід Контролеру.
    - Контролер надсилає хід до Model (HW) через команду REQUEST_CMD::RequestMove.
    - Model (HW) перевіряє результат гри й надсилає відповідь:
        - RESPONSE_CMD::Winner — у разі виявлення переможця.
        - RESPONSE_CMD::Draw — якщо гра завершилася внічию.

- RESPONSE_CMD::NextMove — якщо гра триває, і черга ходу переходить до іншого гравця.

**6.** Оновлення стану (Update State):

- Контролер оновлює поточний стан гри (ігрову дошку, чергу ходу).
- Клієнт отримує нові дані та відображає оновлену дошку на екрані.

**7.** Результат гри (Game Result):

- Якщо відповідь від Model (HW) містить RESPONSE_CMD::Winner, Контролер інформує клієнта про перемогу гравця.
- Якщо відповідь містить RESPONSE_CMD::Draw, Контролер повідомляє про нічию.

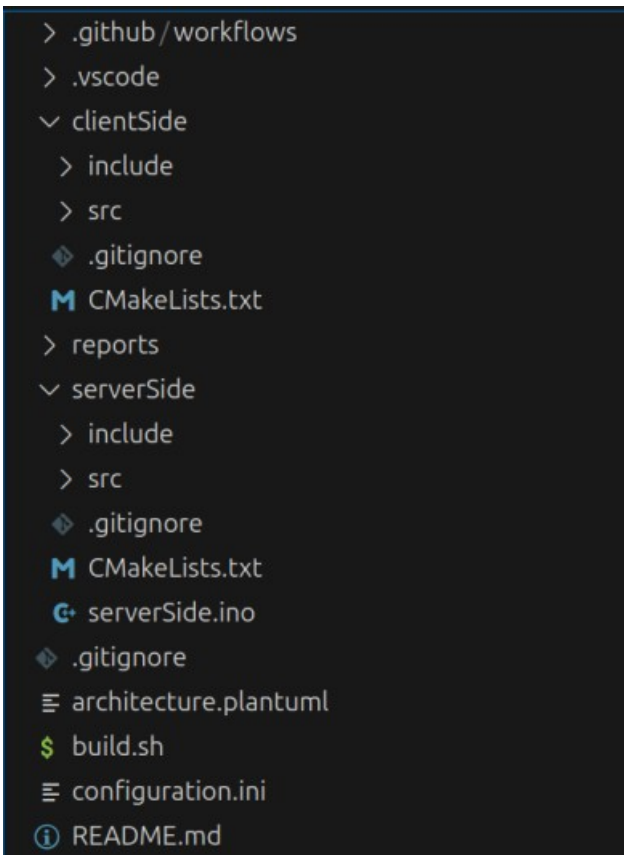



Рис.2. Оновлена структура проекту

Перевірка роботи гри в режимі Man Vs Ai (DEBUG logs enabled):

```
lili@lili-VirtualBox  csad2425ki404khylynskaliliia21 ./clientSide/build/client
Opening port '/dev/ttyUSB0'
Waiting for Arduino to be ready...
Received from Arduino: Arduino ready!

Starting Tic-Tac-Toe game...
Start a new game? (y/n): y
Game configuration loaded successfully.
Message sent: 100_0_X_O_1_123456789$
Response from Arduino: 200

Acknowledgment of game start received from hardware.
-------------
 1 | 2 | 3
-------------
 4 | 5 | 6
-------------
 7 | 8 | 9
-------------
Player 1(X) enter your move (1-9): 5
Message sent: 102_4$
Response from Arduino: 201

-------------
 1 | 2 | 3
-------------
 4 | X | 6
-------------
 7 | 8 | 9
-------------
AI (Player 2(O) chose position 2
Message sent: 102_1$
Response from Arduino: 201
```

```
------------
 1 | O | 3
------------
 4 | X | 6
------------
 7 | 8 | 9
------------
Player 1(X) enter your move (1-9): 1
Message sent: 102_0$
Response from Arduino: 201

------------
 X | O | 3
------------
 4 | X | 6
------------
 7 | 8 | 9
------------
AI (Player 2(O) chose position 8
Message sent: 102_7$
Response from Arduino: 201

------------
 X | O | 3
------------
 4 | X | 6
------------
 7 | O | 9
------------
Player 1(X) enter your move (1-9): 9
Message sent: 102_8$
Response from Arduino: 202

------------
 X | O | 3
------------
 4 | X | 6
------------
 7 | O | X
------------
WE HAVE A WINNER - X !!!
```

Створила утиліту *build.sh* для зручного управління та збірки проєктів клієнта й сервера. Основні можливості та параметри скрипта:

1. Основні параметри:

- client: збирає лише клієнтський проєкт.

- server: збирає лише серверний проєкт. У разі вказання цього параметра додатково виконується завантаження прошивки на сервер.
- clean: видаляє всі створені каталоги збудованих файлів та документацію як для клієнта, так і для сервера.
- -h або --help: відображає довідку щодо використання скрипта.

2. Загальні дії:

- Без зазначення конкретної опції скрипт за замовчуванням виконує збірку обох проєктів — клієнта і сервера.
- Можна вказати додатковий параметр для вибору серійного порту, який буде використано при збірці сервера.

```
lili@lili-VirtualBox  csad2425ki404khylynskaliliia21 bash build.sh -h
Usage: build.sh [OPTION] [SERIAL_PORT]
Build and manage client and server projects.

Options:
  client        Build only the client project.
  server        Build only the server project (includes upload target).
  clean         Remove build directories for both client and server.
  -h, --help    Display this help and exit.

If no option is specified, both client and server will be built.
To specify a custom serial port for the server, pass it as an additional argument.
```

**Висновок:**У ході виконання цієї лабораторної роботи я реалізувала логіку гри для клієнта та сервера, а також налаштувала їх взаємодію за допомогою клієнт-серверної архітектури через UART. Для конфігурації та компіляції клієнтського й серверного проєктів я використала CMake. Також створила скрипт *build.sh* для спрощення процесу збірки.